

Déploiement sur VPS

Ce guide décrit un déploiement simple sur un VPS Linux avec Docker Compose.

Architecture cible

```
VPS uniquement en local
|
v
127.0.0.1:8081
|
v
frontend Nginx React isolé
|
+-- /api/* --> backend FastAPI:8000

Volumes Docker persistants dédiés au projet `rpg40k` :
- rpg40k_data : base SQLite `rpg40k.sqlite3`
- rpg40k_saves : sauvegardes YAML par utilisateur
```

Important — serveur déjà en production

La configuration est volontairement isolée pour ne pas casser un projet déjà en production sur le VPS :

- aucun bind direct sur les ports publics **80** ou **443** ;
- le frontend écoute seulement sur **127.0.0.1:8081** par défaut ;
- l'exposition publique peut se faire sur **0.0.0.0:8081** si besoin, sans toucher aux ports du site existant ;
- Docker Compose est lancé avec le nom de projet **rpg40k** ;
- les volumes sont séparés du reste du serveur ;
- aucune configuration Nginx/Apache/Caddy existante n'est modifiée automatiquement.

Conséquence : après déploiement, l'application est testable depuis le VPS avec **curl http://127.0.0.1:8081**, mais elle n'écrase pas le site public existant.

Pour la rendre publique ensuite, il faudra ajouter manuellement une règle dans le reverse proxy déjà en production, par exemple sur un sous-domaine dédié `rpg.ton-domaine.fr` qui proxy vers `http://127.0.0.1:8081`.

Accès public rapide sans tunnel SSH

Si aucun reverse proxy ou sous-domaine n'est encore prêt, l'application peut être exposée directement sur le port dédié `8081`, sans toucher au site existant sur `80/443`.

Sur le VPS :

```
cd /opt/rpg-40k
grep -q '^RPG40K_BIND_ADDRESS=' .env && sed -i
's/^RPG40K_BIND_ADDRESS=.*RPG40K_BIND_ADDRESS=0.0.0.0/' .env || echo
'RPG40K_BIND_ADDRESS=0.0.0.0' >> .env
grep -q '^RPG40K_HTTP_PORT=' .env && sed -i
's/^RPG40K_HTTP_PORT=.*RPG40K_HTTP_PORT=8081/' .env || echo
'RPG40K_HTTP_PORT=8081' >> .env
docker compose -p rpg40k up -d --build
```

L'application devient accessible ici :

```
http://IP_DU_VPS:8081/
```

Cette solution est utile pour une démonstration. Pour une mise en production propre, préférer ensuite un sous-domaine HTTPS.

Fichiers ajoutés

Fichier	Rôle
Dockerfile.backend	Image backend FastAPI
frontend/Dockerfile	Build React + Nginx
frontend/nginx.conf	Reverse proxy <code>/api</code> vers le backend
docker-compose.yml	Orchestration VPS
.dockerignore	Exclusion secrets/caches

Fichier	Rôle
.env.example	Variables d'environnement
scripts/deploy_vps.sh	Script de déploiement Ubuntu
scripts/backup_vps.sh	Sauvegarde des volumes Docker

Prérequis VPS

- Ubuntu 22.04 ou 24.04 recommandé.
- Accès SSH avec un utilisateur sudo.
- Git installé.
- Docker + Docker Compose v2 installés.
- Port SSH **22** accessible.
- Les ports publics **80** et **443** peuvent déjà être utilisés par l'autre projet en production.
- Le port local **8081** doit seulement être libre sur le VPS.

Installation de Docker sur Ubuntu

```
sudo apt update
sudo apt install -y ca-certificates curl git
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo tee
/etc/apt/keyrings/docker.asc >/dev/null
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(.
/etc/os-release && echo $VERSION_CODENAME) stable" | sudo tee
/etc/apt/sources.list.d/docker.list >/dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
sudo usermod -aG docker $USER
```

Déconnecte-toi puis reconnecte-toi en SSH pour activer le groupe **docker**.

Premier déploiement

```
export APP_DIR=/opt/rpg-40k
export REPO_URL=https://github.com/Berserkyr/rpg-40k.git
export BRANCH=main
curl -fsSL https://raw.githubusercontent.com/Berserkyr/rpg-40k/main/scripts/deploy_vps.sh | bash
```

Puis édite le fichier `.env` sur le VPS :

```
cd /opt/rpg-40k
nano .env
```

Exemple minimal :

```
OPENAI_API_KEY=
VITE_API_BASE=/api
DOMAIN=ton-domaine.fr
APP_DIR=/opt/rpg-40k
RPG40K_BIND_ADDRESS=127.0.0.1
RPG40K_HTTP_PORT=8081
```

Relance ensuite :

```
docker compose -p rpg40k up -d --build
```

Vérification

```
docker compose -p rpg40k ps
curl http://127.0.0.1:8081/api/health
```

Pour tester depuis ton PC sans exposer publiquement l'application, ouvre un tunnel SSH :

```
ssh -L 8081:127.0.0.1:8081 root@IP_DU_VPS
```

Puis ouvre depuis ton PC :

http://127.0.0.1:8081/

Mise à jour applicative

```
cd /opt/rpg-40k
git pull --ff-only origin main
docker compose -p rpg40k up -d --build
```

Déploiement via pipeline

Le déploiement VPS est aussi disponible dans les pipelines CI/CD.

GitHub Actions

Workflow : [.github/workflows/deploy-vps.yml](https://github.com/workflows/deploy-vps.yml)

Déclenchement : manuel via **Actions** → **Deploy VPS** → **Run workflow**.

Secrets GitHub à configurer dans **Settings** → **Secrets and variables** → **Actions** :

Secret / variable	Exemple	Rôle
VPS_HOST	89.116.111.166	Adresse du VPS
VPS_USER	debian	Utilisateur SSH
VPS_SSH_KEY	clé privée SSH dédiée	Clé privée utilisée par GitHub Actions
VPS_PORT	22	Port SSH, optionnel
variable APP_DIR	/opt/rpg-40k	Dossier de déploiement, optionnel

La clé privée ne doit jamais être committée. Créer de préférence une clé dédiée au déploiement et ajouter sa clé publique dans `~/.ssh/authorized_keys` sur le VPS.

GitLab CI

Pipeline : [.gitlab-ci.yml](#)

Job : `deploy-vps`, manuel, uniquement sur `main`.

Variables GitLab CI/CD à configurer :

Variable	Exemple	Rôle
<code>VPS_HOST</code>	<code>89.116.111.166</code>	Adresse du VPS
<code>VPS_USER</code>	<code>debian</code>	Utilisateur SSH
<code>SSH_PRIVATE_KEY</code>	clé privée SSH dédiée	Clé privée utilisée par GitLab
<code>VPS_PORT</code>	<code>22</code>	Port SSH, optionnel
<code>APP_DIR</code>	<code>/opt/rpg-40k</code>	Dossier de déploiement, optionnel
<code>RPG40K_BIND_ADDRESS</code>	<code>0.0.0.0</code>	Exposition publique directe
<code>RPG40K_HTTP_PORT</code>	<code>8081</code>	Port dédié de l'application

Les deux pipelines lancent la même logique : `git pull`, mise à jour du `.env`, `docker compose -p rpg40k up -d --build`, puis vérification de `/api/health`.

Logs

```
docker compose -p rpg40k logs -f backend
docker compose -p rpg40k logs -f frontend
```

Sauvegarde

```
cd /opt/rpg-40k
bash scripts/backup_vps.sh
```

Les archives sont créées dans `~/rpg-40k-backups` par défaut.

HTTPS et nom de domaine

La configuration fournie n'expose pas directement l'application sur le port public **80**.

Pour une mise en production publique sans impacter le site existant, ajouter une règle dans le reverse proxy déjà installé :

1. créer un sous-domaine, par exemple **rpg.ton-domaine.fr** ;
2. le faire pointer vers le VPS ;
3. ajouter une règle proxy vers **http://127.0.0.1:8081** ;
4. générer un certificat TLS ;
5. tester sans modifier le site principal.

Commandes utiles

```
# Arrêter
cd /opt/rpg-40k
docker compose -p rpg40k down

# Redémarrer
cd /opt/rpg-40k
docker compose -p rpg40k restart

# Reconstruire complètement
cd /opt/rpg-40k
docker compose -p rpg40k build --no-cache
docker compose -p rpg40k up -d
```

Points de preuve pour la soutenance

- Déploiement reproductible via **docker-compose.yml**.
- BDD persistée dans un volume Docker.
- Sauvegardes de parties persistées dans un volume Docker.
- Healthcheck backend disponible via **/api/health**.
- Reverse proxy frontend/API documenté.
- Script de sauvegarde exploitable.